The standard also describes how to issue a `WWW-Authenticate` challenge header for bearer tokens, which allows our API to become compliant with the HTTP specifications once again, because you removed that header in chapter 4. The challenge can include a realm parameter, just like any other HTTP authentication scheme, if the API requires different tokens for different endpoints. For example, you might return `realm="users"` from one endpoint and `realm="admins"` from another, to indicate to the client that they should obtain a token from a different login endpoint for administrators compared to regular users. Finally, you can also return a standard error code and description to tell the client why the request was rejected. Of the three error codes defined in the specification, the only one you need to worry about now is `invalid_token`, which indicates that the token passed in the request was expired or otherwise invalid. For example, if a client passed a token that has expired you could return

```
HTTP/1.1 401 Unauthorized
WWW-Authenticate: Bearer realm="users",
error="invalid_token",
        error_description="Token has expired"
```

This lets the client know to re-authenticate to get a new token and then try its request again. Open the TokenController.java file in your editor and update the `validateToken` and logout methods to extract the token from the Authorization header. If the value starts with the string "Bearer" followed by a single space, then you can extract the token ID from the rest of the value. Otherwise you should ignore it, to allow HTTP Basic authentication to still